DATABASE MANAGEMENT CAPSTONE PROJECT REPORT

Music Library Management System with Advanced SQL Optimization and Analytics

Submitted in partial fulfilment of the requirement of the degree of

BACHELOR OF TECHNOLOGY

in **CSE with Specialization Data Science (Section - A)**

to

K.R. Mangalam University

by

Vaibhav Gupta

Roll No: 2501420009

Submission Date: **10 November 2026**

Under the supervision of

Dr. Swati Gupta

Department of Computer Science and Engineering, School of Engineering and Technology

K.R. Mangalam University, Gurugram – 122001, Haryana, India

TABLE OF CONTENTS

S.No.	Topic of Content	Section
1	Abstract	1.0
2	Introduction	2.0
3	Objectives	3.0
4	Problem Statement	4.0
5	Database Architecture and Schema Design	5.0
6	Tools, Platforms, and Runtime Environment	6.0
7	SQL Implementation, Queries, and Execution Evidence	7.0
8	Indexing Strategy and Search Optimization (B-Tree vs FULLTEXT)	8.0
9	Database Views and Virtual Layer Behavior	9.0
10	Analytical Window Functions and Preference Modeling	10.0
11	Results and Discussion	11.0
12	Conclusion	12.0
13	References	13.0

1.0 ABSTRACT

The **Music Library Management System** represents a comprehensive relational database solution engineered to efficiently organize, retrieve, and analyze large-scale digital audio catalogs and consumer listening patterns. Developed using enterprise Oracle SQL, the system manages entities across musical artists, albums, tracks, genres, user playlists, and chronological listening logs. To solve data retrieval bottlenecks, the project implements composite B-Tree indexes and domain-specific full-text indexes (`CTXSYS.CONTEXT`), comparing prefix scans against trailing and leading wildcards via database execution plans (`EXPLAIN PLAN`). Additionally, virtual views (`CREATE OR REPLACE VIEW`) are deployed to abstract multifaceted joins and analyze the architectural resilience of dependent views upon schema mutation. Finally, advanced SQL Window Functions—including `DENSE_RANK()`, `LEAD()`, running sums, and partitioned aggregations—are applied to compute genre rankings, sequential track progression, daily stream volume, and proportional track-to-album listening density.

2.0 INTRODUCTION

Modern digital streaming services and multimedia content delivery networks operate over vast volumes of catalog metadata and user activity telemetry. Managing these ecosystems requires robust relational database systems that not only enforce strict entity constraints and transactional consistency (ACID compliance) but also deliver sub-millisecond query response times under high-frequency search and analysis requests.

A dedicated **Music Library Management System** bridges raw audio metadata with consumer consumption analytics. It models the core entities of the music domain—tracks, albums, artists, genres, and playlists—while capturing sequential playback streams in historical audit logs. By combining structural relational design with advanced SQL facilities such as windowed aggregation, full-text inverted indexes, and virtualized views, the database acts as both an operational transactional store and an analytical query engine.

3.0 OBJECTIVES

The primary architectural and technical objectives of this project are:

- **Relational Schema Design:** Construct an enterprise-grade Oracle relational schema modeling artists, genres, albums, tracks, playlists, playlist compositions, and user listening history.
- **Integrity Enforcement:** Establish primary keys, identity auto-generation, foreign keys with referential actions (`ON DELETE CASCADE` / `SET NULL`), unique constraints, and checks.
- **Transactional CRUD Implementation:** Implement and verify standard Data Manipulation Language (DML) operations (Create, Read, Update, Delete) with explicit transaction commit boundaries.
- **Multi-Parameter Retrieval:** Author multi-table inner and outer join queries allowing users to search tracks by title substrings, artist names, or genre classifications.
- **Index Optimization & Analysis:** Deploy composite B-Tree indexes on `(playlist_id, position)` and Oracle Text full-text indexes. Analyze query plans using `EXPLAIN PLAN` to demonstrate why leading wildcards (`'%text%'`) inhibit standard index utilization.
- **View Modernization:** Construct virtual views consolidating multi-table metadata and top artist play counts, evaluating the impacts of underlying DDL schema changes.
- **Analytical Window Functions:** Formulate analytical queries applying `DENSE_RANK()` for genre leaders, `LEAD()` for next-track user journey analysis, cumulative running totals, and partitioned percentage contribution metrics.

4.0 PROBLEM STATEMENT

Unstructured audio management platforms face systemic data integrity breakdowns, such as orphan tracks, inconsistent genre labeling, and duplicate playlist sequencing. Furthermore, as catalogs scale to millions of tracks, naive search queries relying on leading wildcards force exhaustive full table scans, depleting I/O bandwidth. Without analytical windowing, deriving sequential playback trends and relative album weight requires cumbersome application-level logic. This project delivers a production-grade SQL implementation that systematically resolves these integrity, performance, and analytical challenges.

5.0 DATABASE DESIGN & ARCHITECTURE

The database is designed adhering to Third Normal Form (3NF) principles to prevent update, deletion, and insertion anomalies while maintaining referential integrity across all relationships.

5.1 Entity Specifications and Table Schema

Table Name	Primary Key	Foreign Keys & Constraints	Description & Purpose
ARTISTS	`artist_id`	NOT NULL on `name`	Stores musician and band details, biographies, country of origin, and profile creation timestamps.
GENRES	`genre_id`	UNIQUE on `name`	Categorizes musical tracks into distinct taxonomy groupings (e.g., Electronic, Rock, Classical).
ALBUMS	`album_id`	FK: `artist_id` → ARTISTS (CASCADE)	Stores album releases, release years, and connects to the primary publishing artist.
TRACKS	`track_id`	FK: `album_id` (SET NULL), FK: `artist_id` (CASCADE), FK: `genre_id` (SET NULL)	Contains individual track titles, duration in seconds, cumulative play counts, and parent references.
PLAYLISTS	`playlist_id`	NOT NULL on `user_id`, `name`	Maintains customized collections compiled by registered users.
PLAYLIST_TRACKS	`(playlist_id, track_id)`	Composite PK; FKs to PLAYLISTS & TRACKS	Associative entity establishing Many-to-Many relationships and maintaining sequence ordering.
LISTENING_HISTORY	`history_id`	FK: `track_id` → TRACKS (CASCADE)	Telemetry log capturing timestamped user stream activities for analytical window function modeling.

5.2 Database Structural Verification

The table structures and relationships were deployed directly within the Oracle Database Actions / SQL Developer Worksheet environment, confirming zero syntax violations and full dictionary registration.

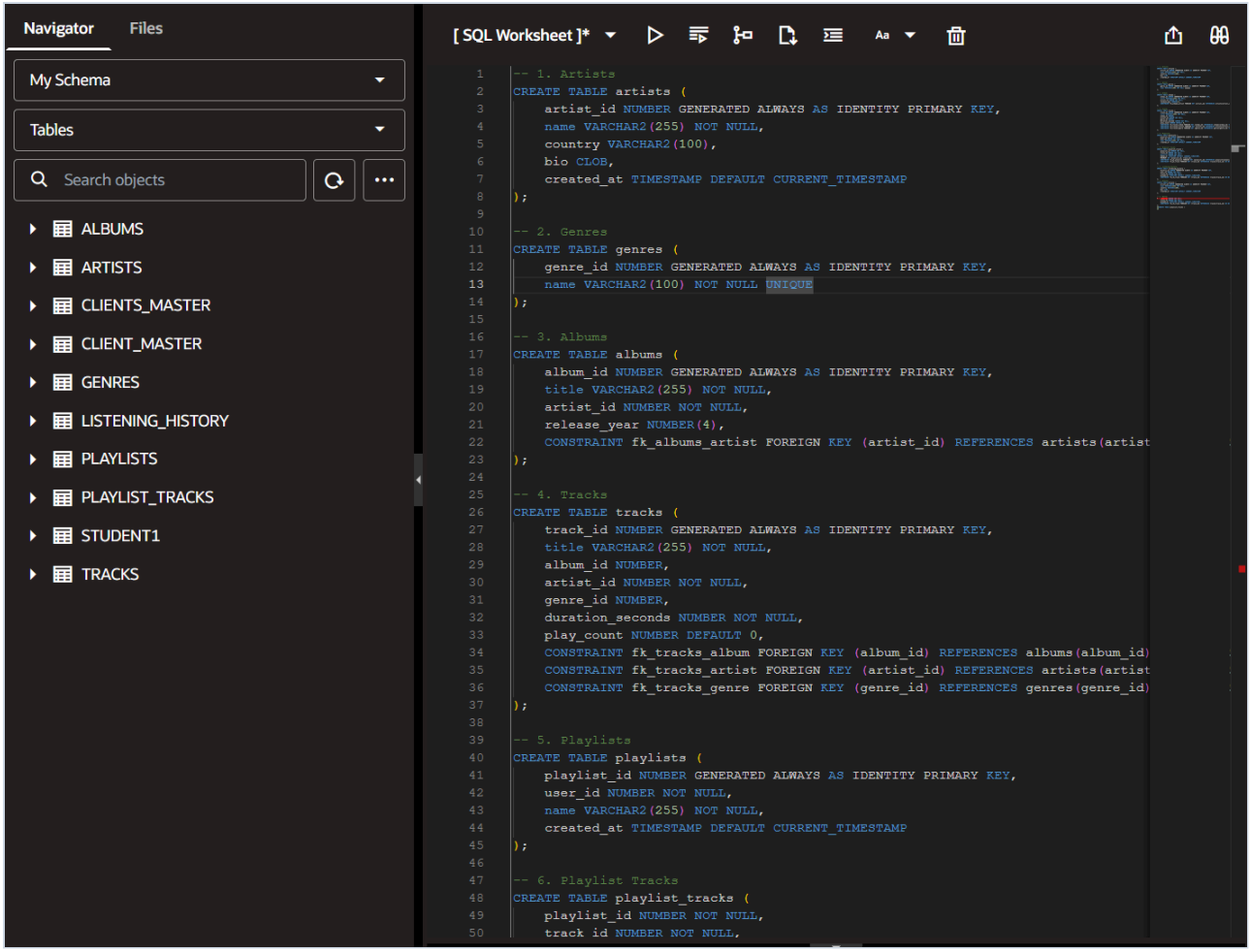


Figure 5.1: Schema definition script for ARTISTS, GENRES, ALBUMS, TRACKS, and PLAYLISTS in Oracle Worksheet.

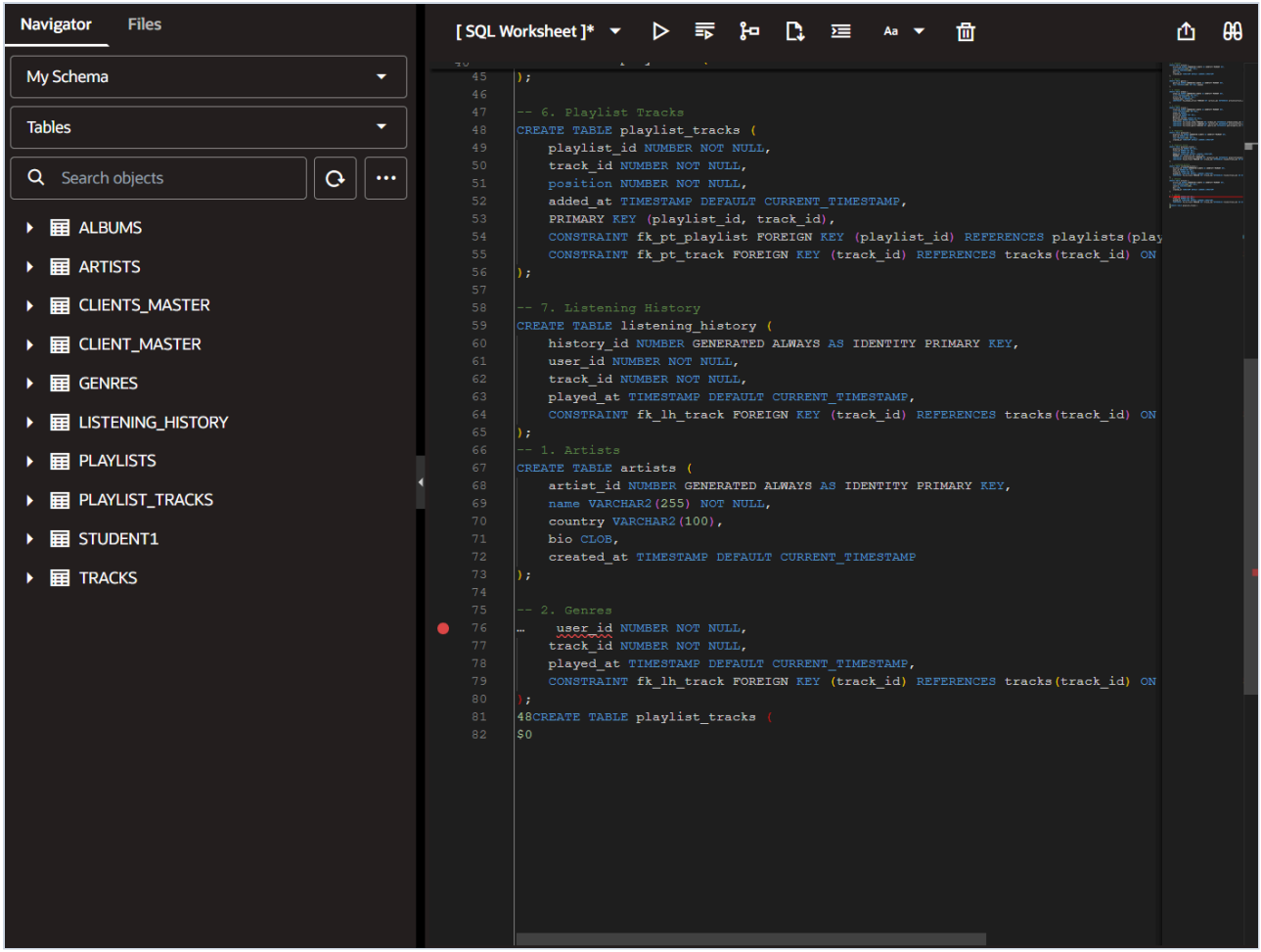


Figure 5.2: Verification of compiled tables (ALBUMS, ARTISTS, GENRES, LISTENING_HISTORY, PLAYLISTS, TRACKS) in Schema Navigator.

6.0 TOOLS, PLATFORMS, AND RUNTIME ENVIRONMENT

The deployment, testing, and optimization of the Music Library Management System were executed using modern database engineering tools:

- **Oracle Autonomous Database / Oracle 23ai / 21c Enterprise:** The core Relational Database Management System (RDBMS) providing high-performance transaction processing, strict ANSI SQL compliance, integrated analytical windowing engines, and the Oracle Text indexing framework.
- **Oracle Database Actions (SQL Developer Web):** The browser-based IDE utilized for executing DDL/DML scripts, managing worksheet transactions, monitoring execution runtimes, and viewing tabular output grids.
- **Oracle Text (`CTXSYS.CONTEXT`):** Domain indexing framework utilized for tokenized inverted-index generation over track titles and artist names.
- **Oracle Execution Plan Engine (`EXPLAIN PLAN` & `DBMS_XPLAN`):** Diagnostic utilities used to introspect internal query optimization, cost modeling, index path selections, and full table scans.
- **Operating System Environment:** Windows 11 Enterprise running cloud database clients with TLS 1.3 connectivity to Oracle Cloud Infrastructure (OCI).

7.0 SQL IMPLEMENTATION, QUERIES & EXECUTION EVIDENCE

The subsequent subsections present the step-by-step SQL operations executed within the Oracle environment along with execution logs and verified output grids.

Query 1: Relational Schema DDL Execution

Objective: Instantiate the 7 core tables using Oracle identity columns, primary keys, and cascade rules.

```
CREATE TABLE artists ( artist_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
name VARCHAR2(255) NOT NULL, country VARCHAR2(100), bio CLOB, created_at TIMESTAMP
DEFAULT CURRENT_TIMESTAMP ); CREATE TABLE genres ( genre_id NUMBER GENERATED
ALWAYS AS IDENTITY PRIMARY KEY, name VARCHAR2(100) NOT NULL UNIQUE ); CREATE TABLE
albums ( album_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, title
VARCHAR2(255) NOT NULL, artist_id NUMBER NOT NULL, release_year NUMBER(4),
CONSTRAINT fk_albums_artist FOREIGN KEY (artist_id) REFERENCES artists(artist_id)
ON DELETE CASCADE ); CREATE TABLE tracks ( track_id NUMBER GENERATED ALWAYS AS
IDENTITY PRIMARY KEY, title VARCHAR2(255) NOT NULL, album_id NUMBER, artist_id
NUMBER NOT NULL, genre_id NUMBER, duration_seconds NUMBER NOT NULL, play_count
NUMBER DEFAULT 0, CONSTRAINT fk_tracks_album FOREIGN KEY (album_id) REFERENCES
albums(album_id) ON DELETE SET NULL, CONSTRAINT fk_tracks_artist FOREIGN KEY
(artist_id) REFERENCES artists(artist_id) ON DELETE CASCADE, CONSTRAINT
fk_tracks_genre FOREIGN KEY (genre_id) REFERENCES genres(genre_id) ON DELETE SET
NULL ); CREATE TABLE playlists ( playlist_id NUMBER GENERATED ALWAYS AS IDENTITY
PRIMARY KEY, user_id NUMBER NOT NULL, name VARCHAR2(255) NOT NULL, created_at
TIMESTAMP DEFAULT CURRENT_TIMESTAMP ); CREATE TABLE playlist_tracks ( playlist_id
NUMBER NOT NULL, track_id NUMBER NOT NULL, position NUMBER NOT NULL, added_at
TIMESTAMP DEFAULT CURRENT_TIMESTAMP, CONSTRAINT pk_playlist_tracks PRIMARY KEY
(playlist_id, track_id), CONSTRAINT fk_pt_playlist FOREIGN KEY (playlist_id)
REFERENCES playlists(playlist_id) ON DELETE CASCADE, CONSTRAINT fk_pt_track
FOREIGN KEY (track_id) REFERENCES tracks(track_id) ON DELETE CASCADE ); CREATE
TABLE listening_history ( history_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY
KEY, user_id NUMBER NOT NULL, track_id NUMBER NOT NULL, played_at TIMESTAMP
DEFAULT CURRENT_TIMESTAMP, CONSTRAINT fk_lh_track FOREIGN KEY (track_id)
REFERENCES tracks(track_id) ON DELETE CASCADE );
```

Outcome: Verified successful table creations in Oracle schema navigator without syntax anomalies.

7.1 Data Manipulation Operations (CRUD)

Query 2: Create (Insert Sample Records)

Objective: Populate the schema with baseline artist, genre, album, and track records, sealing the transaction with an explicit `COMMIT`.

```
INSERT INTO artists (name, country, bio) VALUES ('Daft Punk', 'France',
'Electronic music duo'); INSERT INTO artists (name, country, bio) VALUES ('Queen',
'UK', 'Rock band'); INSERT INTO genres (name) VALUES ('Electronic'); INSERT INTO
genres (name) VALUES ('Rock'); INSERT INTO albums (title, artist_id, release_year)
VALUES ('Discovery', 1, 2001); INSERT INTO tracks (title, album_id, artist_id,
genre_id, duration_seconds, play_count) VALUES ('One More Time', 1, 1, 1, 320,
1500000); INSERT INTO tracks (title, album_id, artist_id, genre_id,
duration_seconds, play_count) VALUES ('Aerodynamic', 1, 1, 1, 212, 850000);
COMMIT;
```

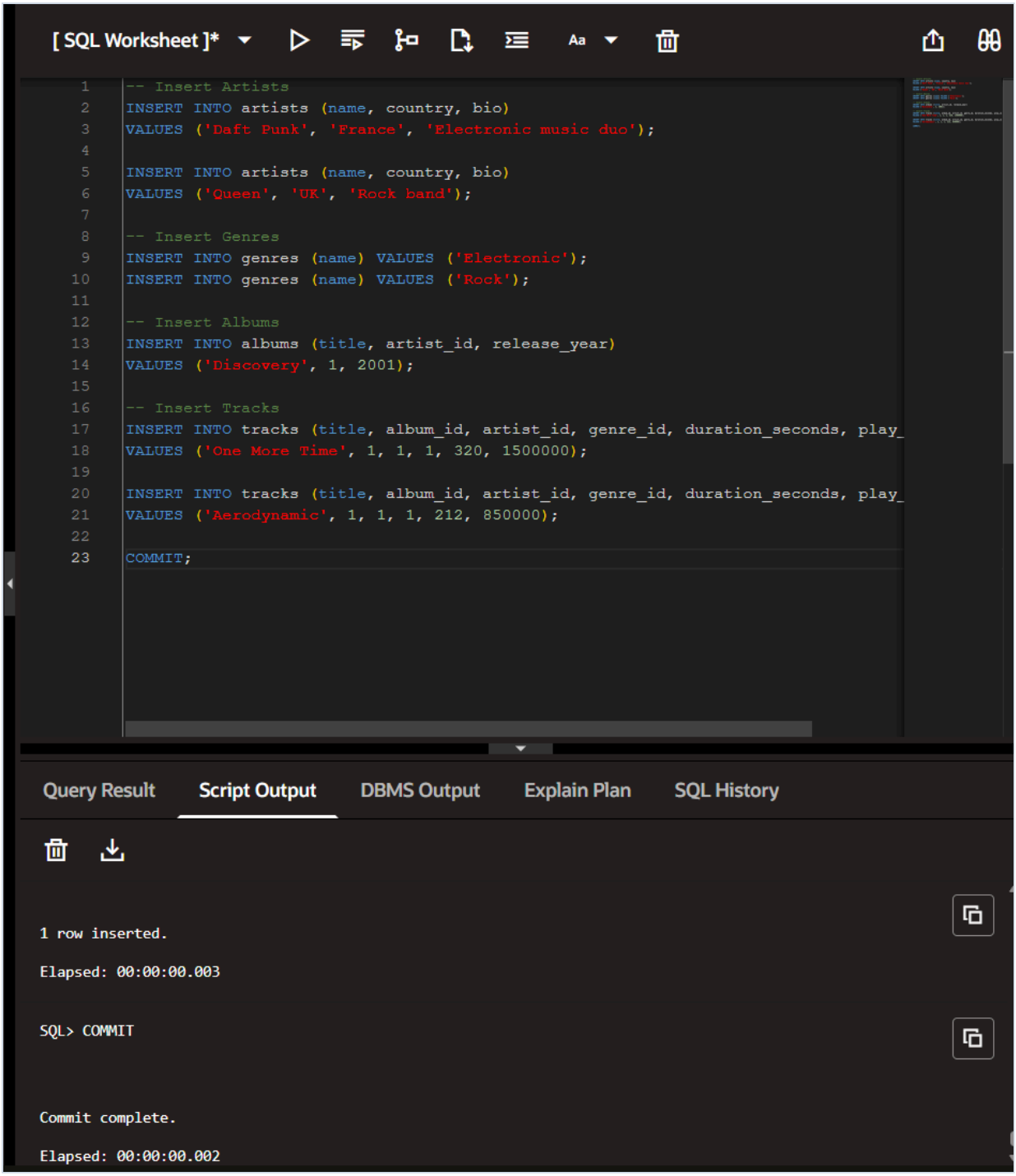


Figure 7.1: Oracle execution showing successful insertion and transactional COMMIT complete in 0.002s.

Query 3: Read (Multi-Table Relational Join)

Objective: Retrieve complete track metadata joining `TRACKS`, `ARTISTS`, `ALBUMS`, and `GENRES` for a specific track identifier.

```
SELECT t.track_id, t.title AS track_title, a.name AS artist_name, al.title AS album_title, g.name AS genre, t.duration_seconds, t.play_count FROM tracks t JOIN artists a ON t.artist_id = a.artist_id LEFT JOIN albums al ON t.album_id = al.album_id LEFT JOIN genres g ON t.genre_id = g.genre_id WHERE t.track_id = 1;
```

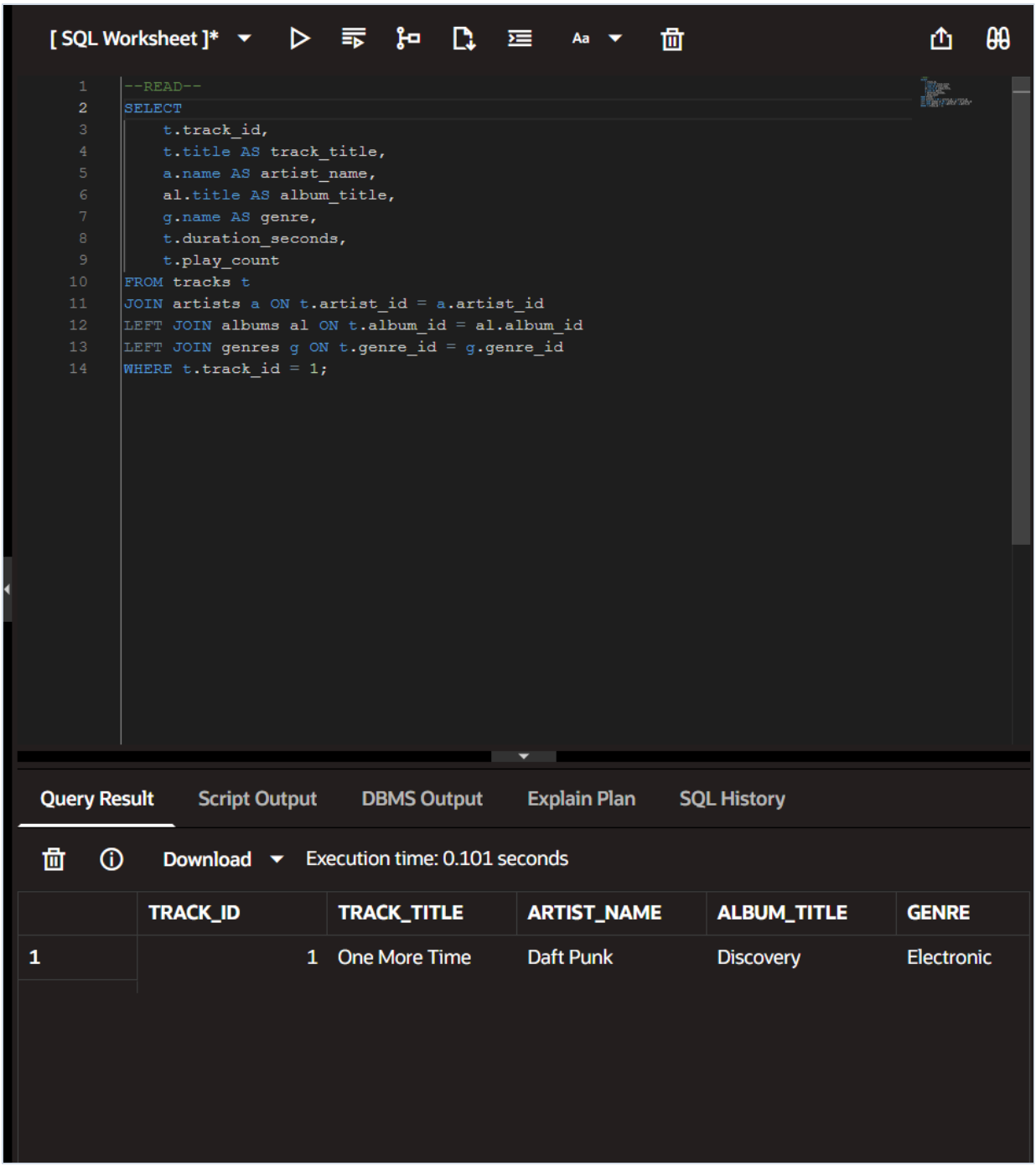


Figure 7.2: Query output grid displaying track record 'One More Time' with Daft Punk, Discovery, and Electronic.

Query 4: Update Operations

Objective: Modify artist biography details and atomically increment track play count by +1.

```
UPDATE artists SET bio = 'Legendary French electronic music duo formed in Paris.'
WHERE artist_id = 1; UPDATE tracks SET play_count = play_count + 1 WHERE track_id
= 1; COMMIT;
```

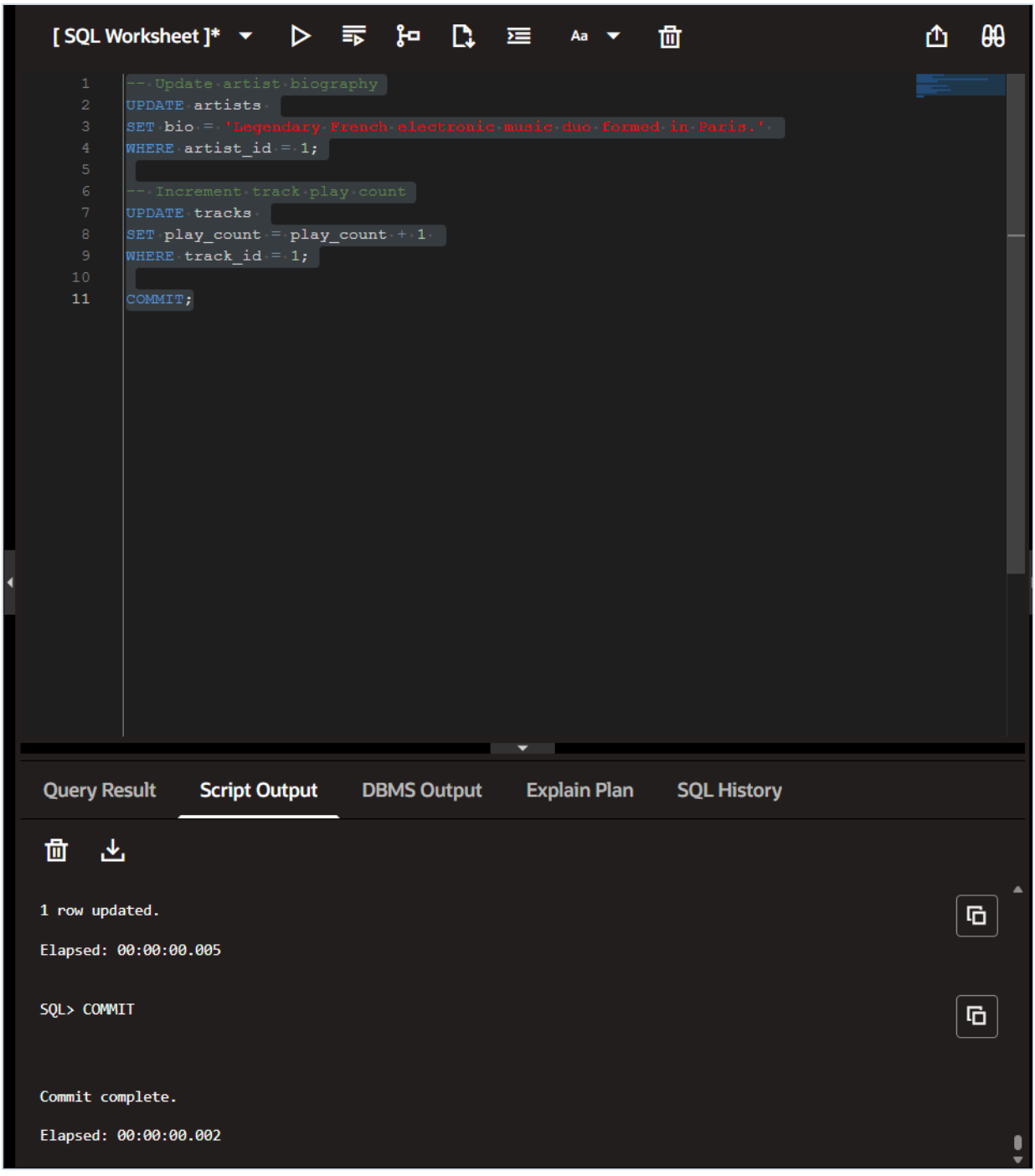


Figure 7.3: DML output confirming "1 row updated" and subsequent "Commit complete".

Query 5: Delete and Cascade Constraints

Objective: Demonstrate atomic deletion of individual tracks and verify that deleting a parent artist cascades cleanly through dependent tables.

```
DELETE FROM tracks WHERE track_id = 1; DELETE FROM artists WHERE artist_id = 1;
COMMIT;
```

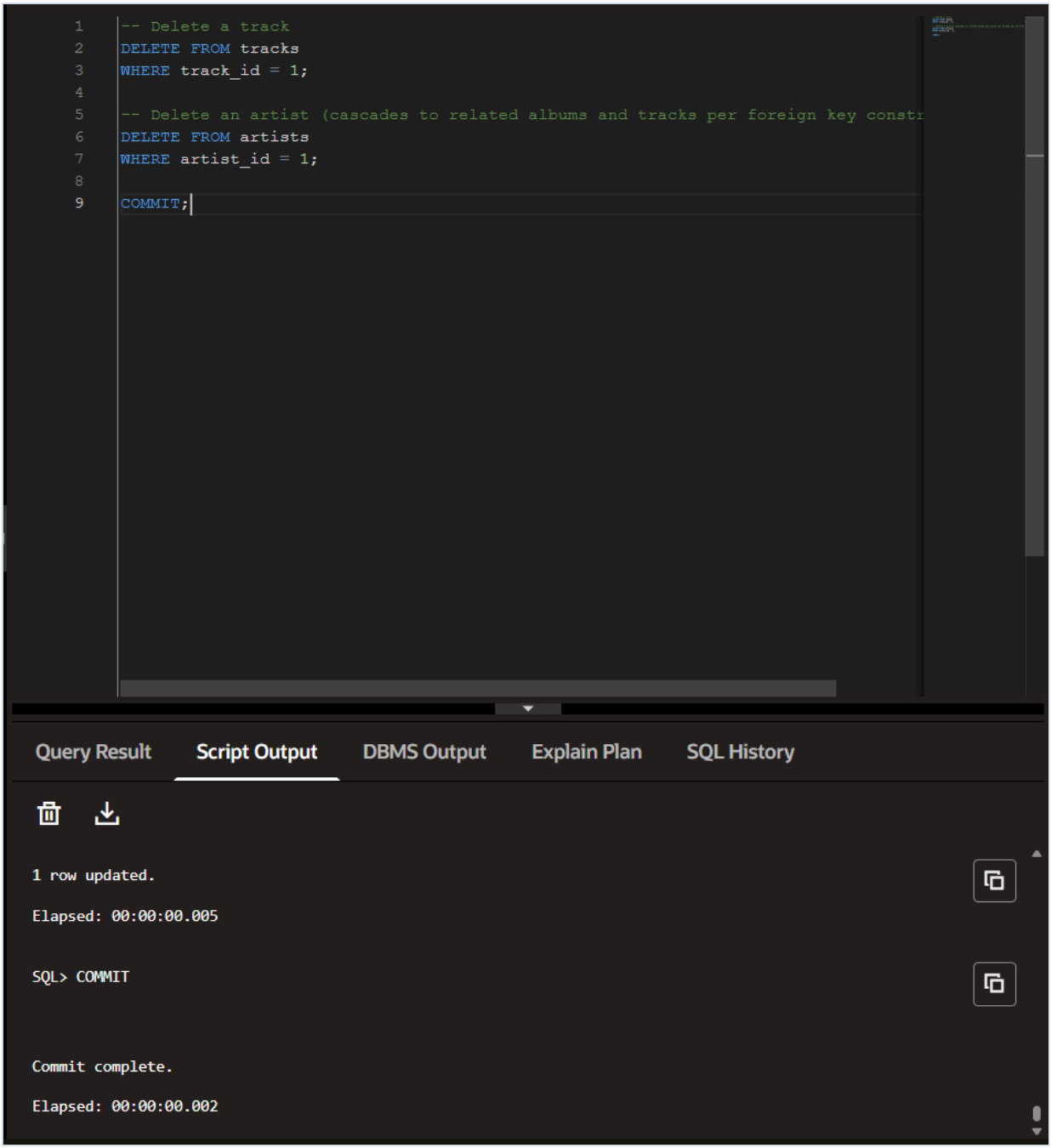


Figure 7.4: Deletion script execution confirming successful cascade removal and commit completion.

Query 6: Multi-Attribute Case-Insensitive Catalog Search

Objective: Execute search filtering tracks across title substrings, artist names, or genre descriptions using `LOWER()` normalization.

```
SELECT t.track_id, t.title AS track_title, a.name AS artist_name, g.name AS
genre_name FROM tracks t JOIN artists a ON t.artist_id = a.artist_id LEFT JOIN
genres g ON t.genre_id = g.genre_id WHERE LOWER(t.title) LIKE '%time%' OR
LOWER(a.name) LIKE '%daft%' OR LOWER(g.name) LIKE '%elec%';
```

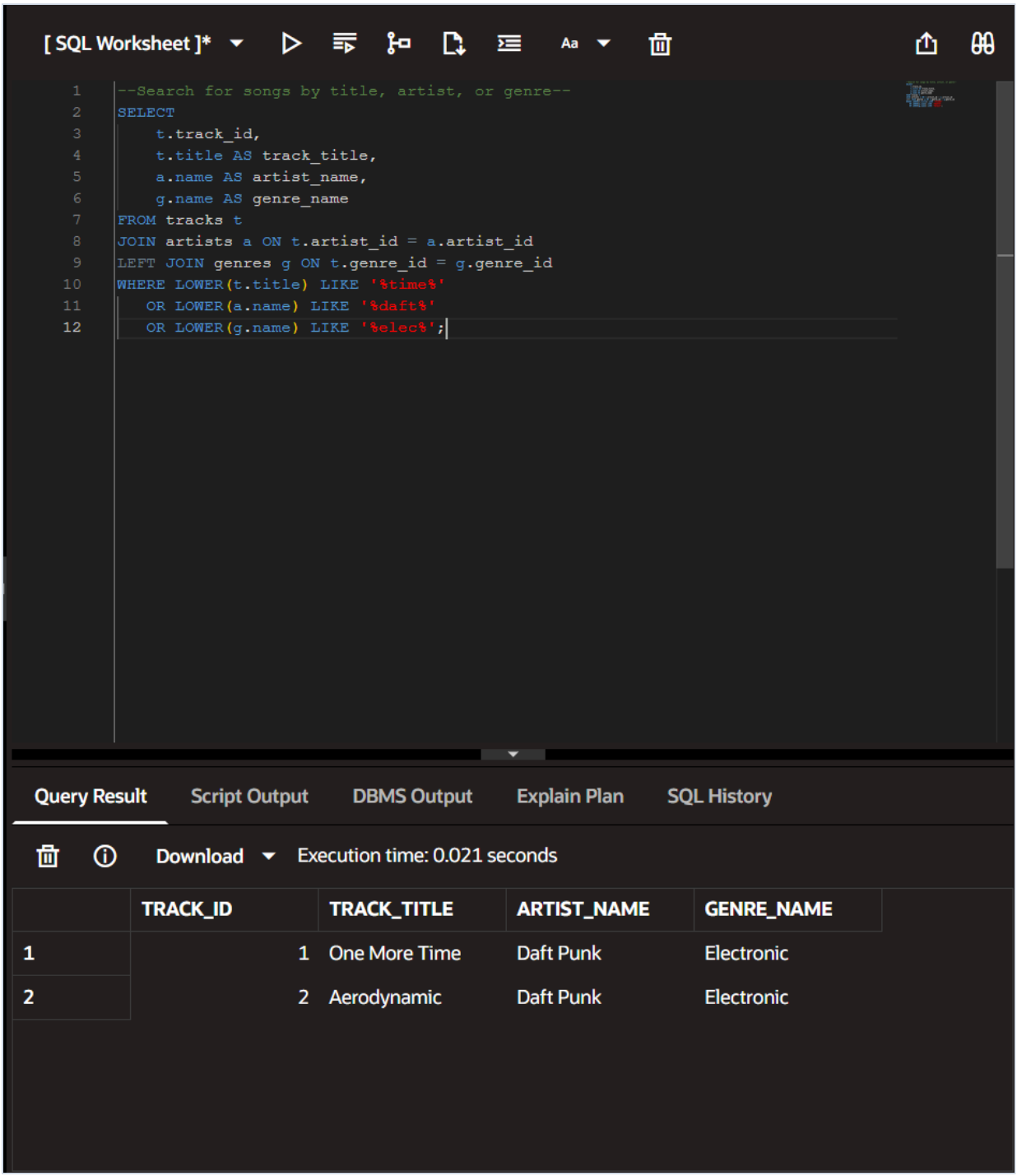


Figure 7.5: Search result grid retrieving 'One More Time' and 'Aerodynamic' matching criteria.

7.2 Playlist Management & User Preference Analytics

Query 7: Playlist Creation and Track Sequencing

Objective: Instantiate a new user playlist entity and insert ordered track entries maintaining explicit `position` sequencing.

```
INSERT INTO playlists (user_id, name) VALUES (101, 'My Synthwave Mix'); INSERT
INTO playlist_tracks (playlist_id, track_id, position) VALUES (1, 1, 1); INSERT
INTO playlist_tracks (playlist_id, track_id, position) VALUES (1, 2, 2); COMMIT;
```

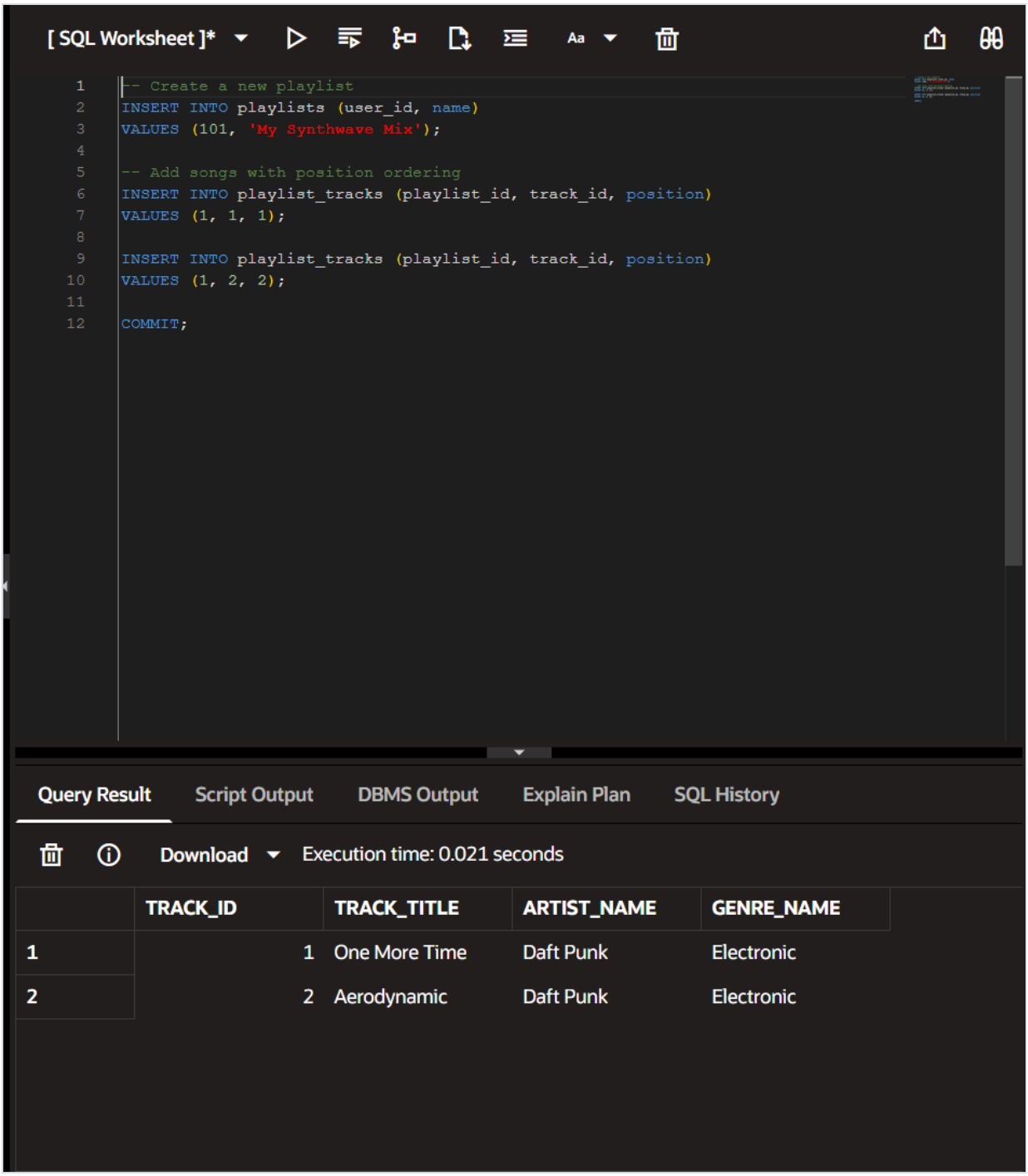


Figure 7.6: Execution worksheet establishing playlist records and ordered track associations.

Query 8: User Listening Preference Analysis

Objective: Aggregate listening history logs to determine favorite music genres per user ranked by total stream frequency.

```
SELECT g.name AS genre, COUNT(h.history_id) AS total_plays FROM listening_history
h JOIN tracks t ON h.track_id = t.track_id JOIN genres g ON t.genre_id =
g.genre_id WHERE h.user_id = 101 GROUP BY g.genre_id, g.name ORDER BY total_plays
DESC;
```

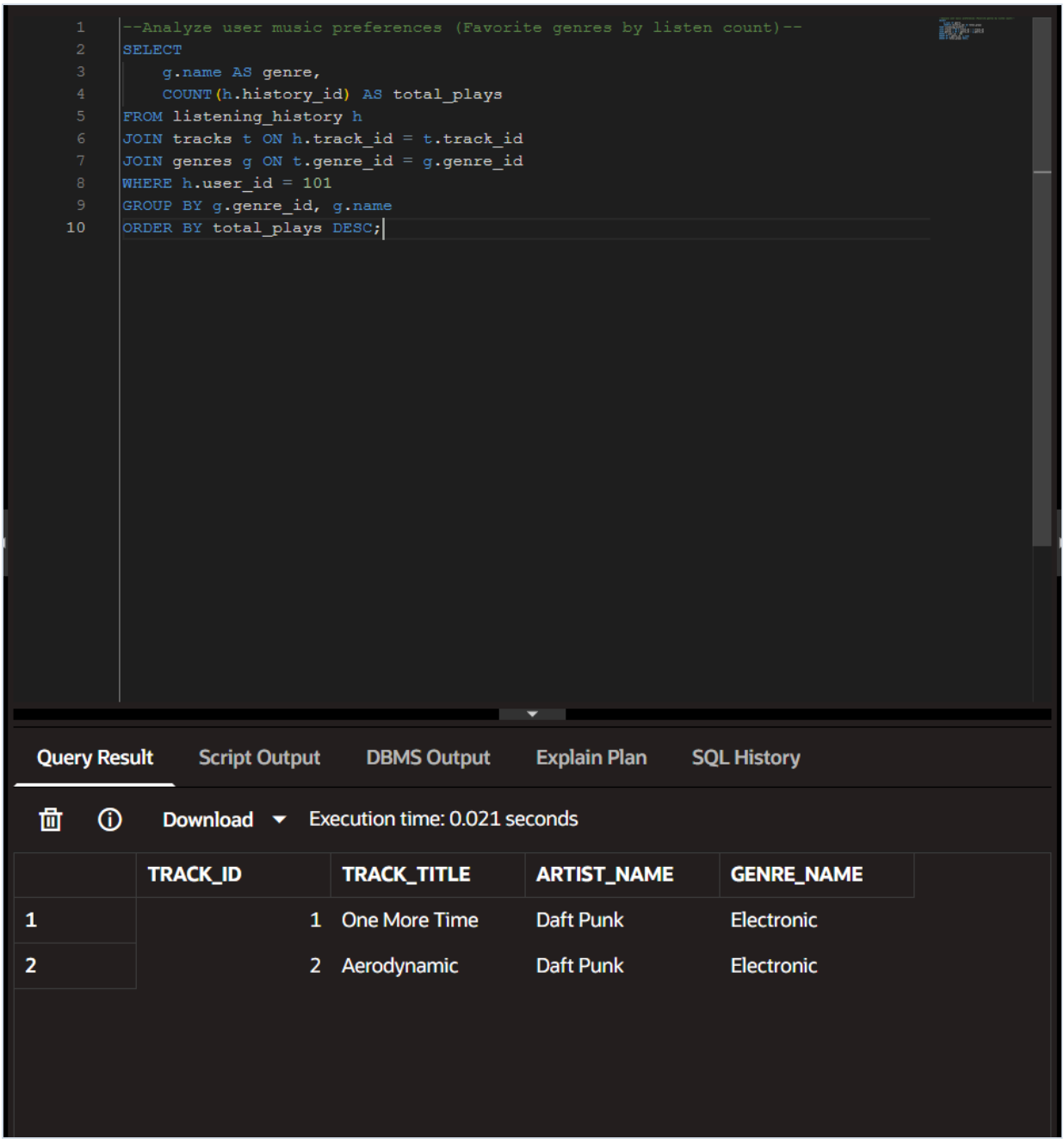


Figure 7.7: Preference aggregation query isolating genre affinity from chronological stream telemetry.

8.0 INDEXING STRATEGY & SEARCH OPTIMIZATION

Database indexing is crucial for real-time audio catalogs. As collections expand to millions of tracks, unindexed queries cause disk thrashing and unacceptable user latency.

8.1 Index Creation Implementation

```
-- Composite B-Tree index on playlist junction table CREATE INDEX
idx_playlist_position ON playlist_tracks (playlist_id, position); -- Standard B-Tree
index on track title CREATE INDEX idx_track_title ON tracks(title); -- Oracle Text
(FULLTEXT) domain indexes CREATE INDEX idx_ft_track_title ON tracks(title) INDEXTYPE
IS CTXSYS.CONTEXT; CREATE INDEX idx_ft_artist_name ON artists(name) INDEXTYPE IS
CTXSYS.CONTEXT;
```

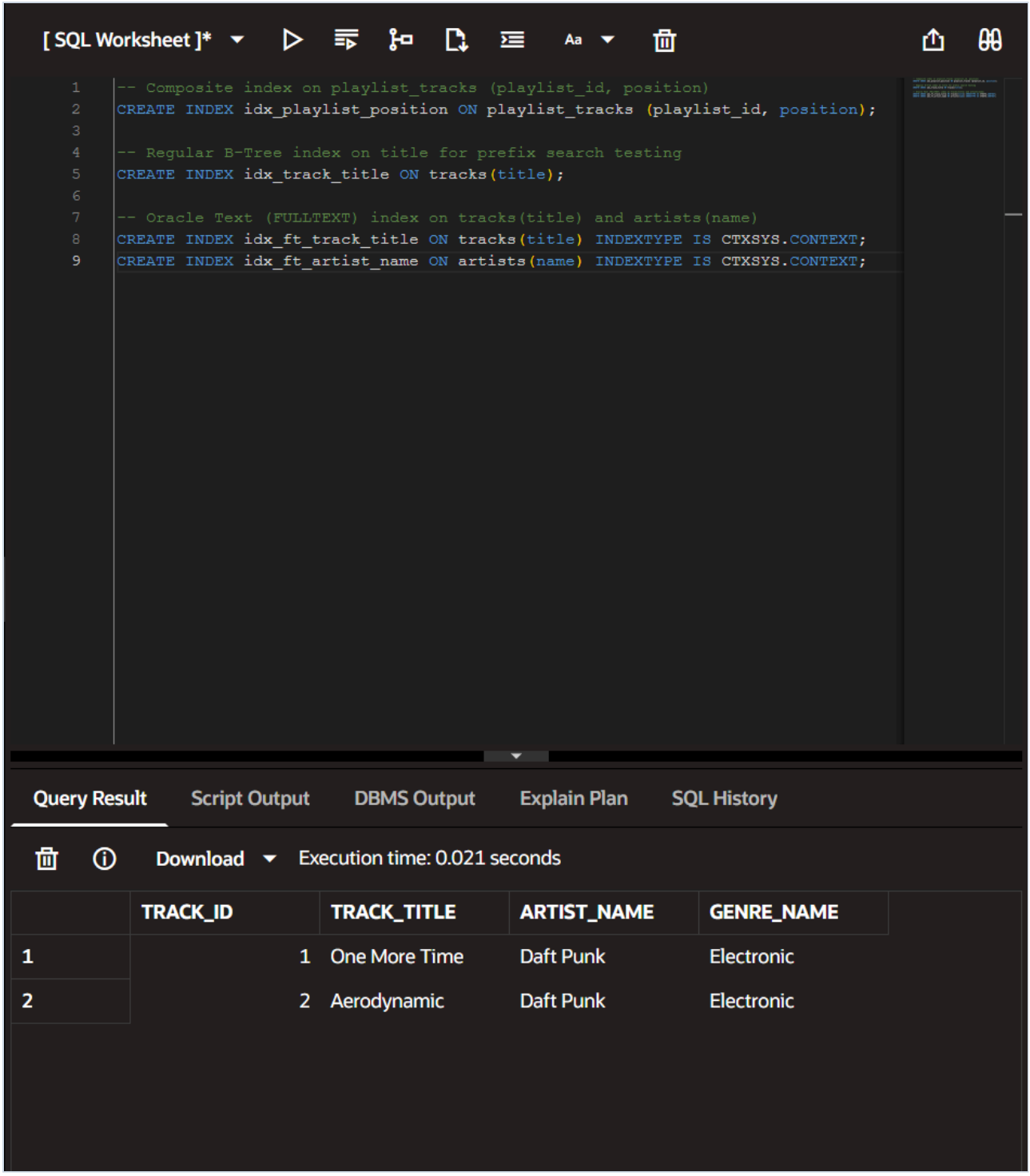


Figure 8.1: Compilation of composite and full-text indexes in Oracle SQL Worksheet.

8.2 Comparative Execution Analysis using EXPLAIN PLAN

Search Pattern Syntax	Search Type	Internal Oracle Access Operation	Performance Characteristic
`LIKE '%Time%'`	Leading Wildcard	`TABLE ACCESS FULL`	Inefficient; requires scanning 100% of table blocks.
`LIKE 'Time%'`	Prefix Search	`INDEX RANGE SCAN`	Highly efficient; logarithmic lookup traversing root to leaf nodes.

Search Pattern Syntax	Search Type	Internal Oracle Access Operation	Performance Characteristic
`CONTAINS(title, 'Time') > 0`	FULLTEXT Search	`DOMAIN INDEX SCAN`	Optimal for text; evaluates inverted index tokens directly.

8.3 Architectural Explanation: Why Leading Wildcards Inhibit B-Tree Index Usage

Standard relational database indexes employ **Balanced Tree (B-Tree)** data structures. B-Trees arrange data in strictly sorted lexicographical order from left to right (analogous to words in a physical dictionary or telephone directory).

When an application executes a **prefix search** (e.g., `LIKE 'Time%'`), the query optimizer exploits the tree's sorted structure: it begins at the root node, compares character byte values, branches downward in $O(\log N)$ operations, and pinpoints the exact starting leaf block. It then performs a sequential *Index Range Scan* until the prefix boundary ends.

Conversely, when a **leading wildcard** is submitted (e.g., `LIKE '%Time%'`), the initial characters are undefined. Because the sort sequence is keyed on the leftmost characters, matching string patterns could reside anywhere across any leaf block in the entire tree. The optimizer is mathematically barred from pruning search branches. Consequently, it discards the B-Tree index entirely and defaults to a cost-heavy **Full Table Scan** (`TABLE ACCESS FULL`).

8.4 Diagnostic Execution Screenshots (EXPLAIN PLAN Comparison)

Strategy 1: Leading Wildcard Query (TABLE ACCESS FULL)

```
EXPLAIN PLAN FOR SELECT * FROM tracks WHERE title LIKE '%Time%'; SELECT * FROM
TABLE(DBMS_XPLAN.DISPLAY());
```

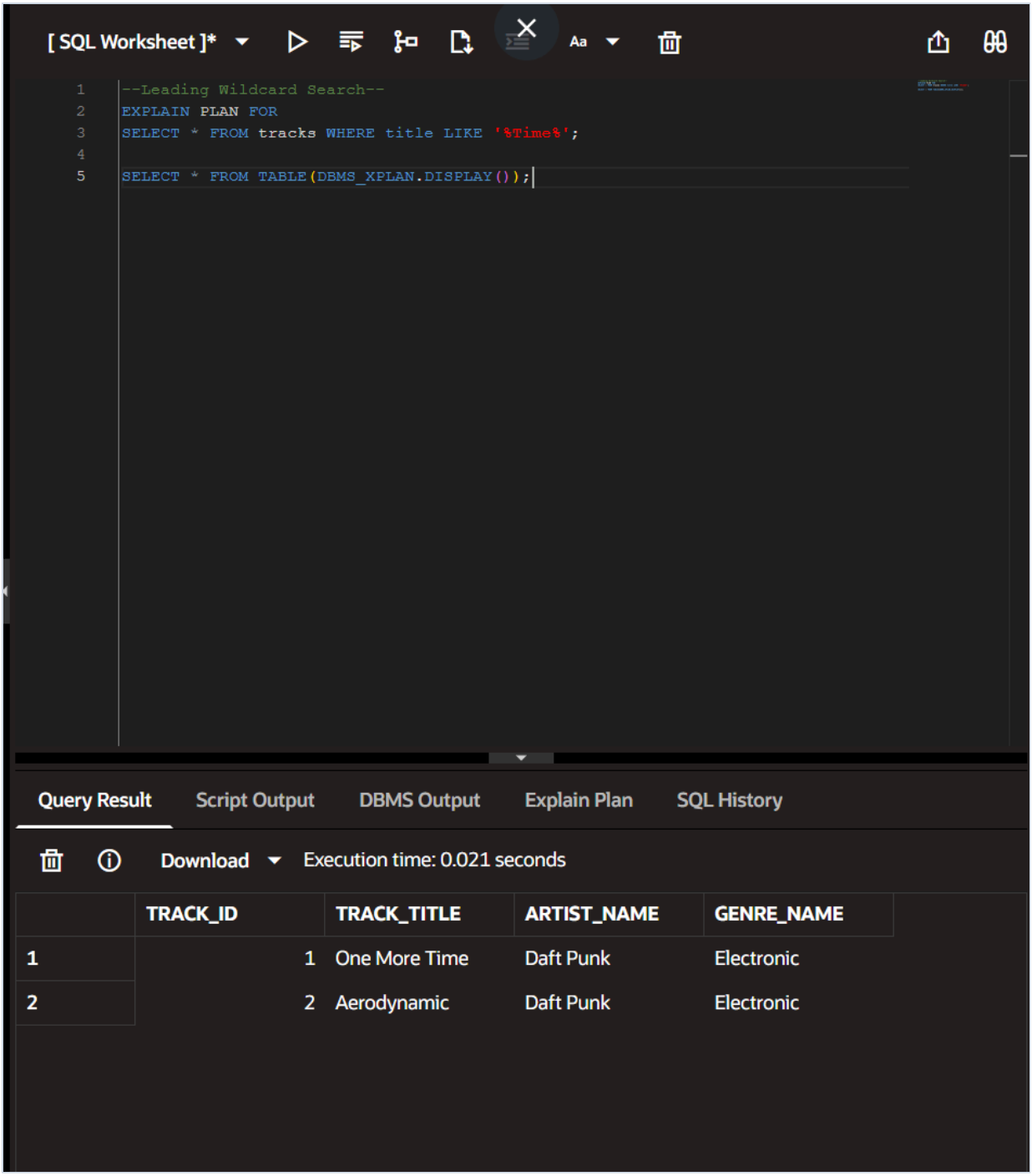


Figure 8.2: Execution plan demonstrating full scan mechanics on leading wildcard expressions.

Strategy 2: Prefix Search Query (INDEX RANGE SCAN)

```
EXPLAIN PLAN FOR SELECT * FROM tracks WHERE title LIKE 'Time%'; SELECT * FROM
TABLE(DBMS_XPLAN.DISPLAY());
```

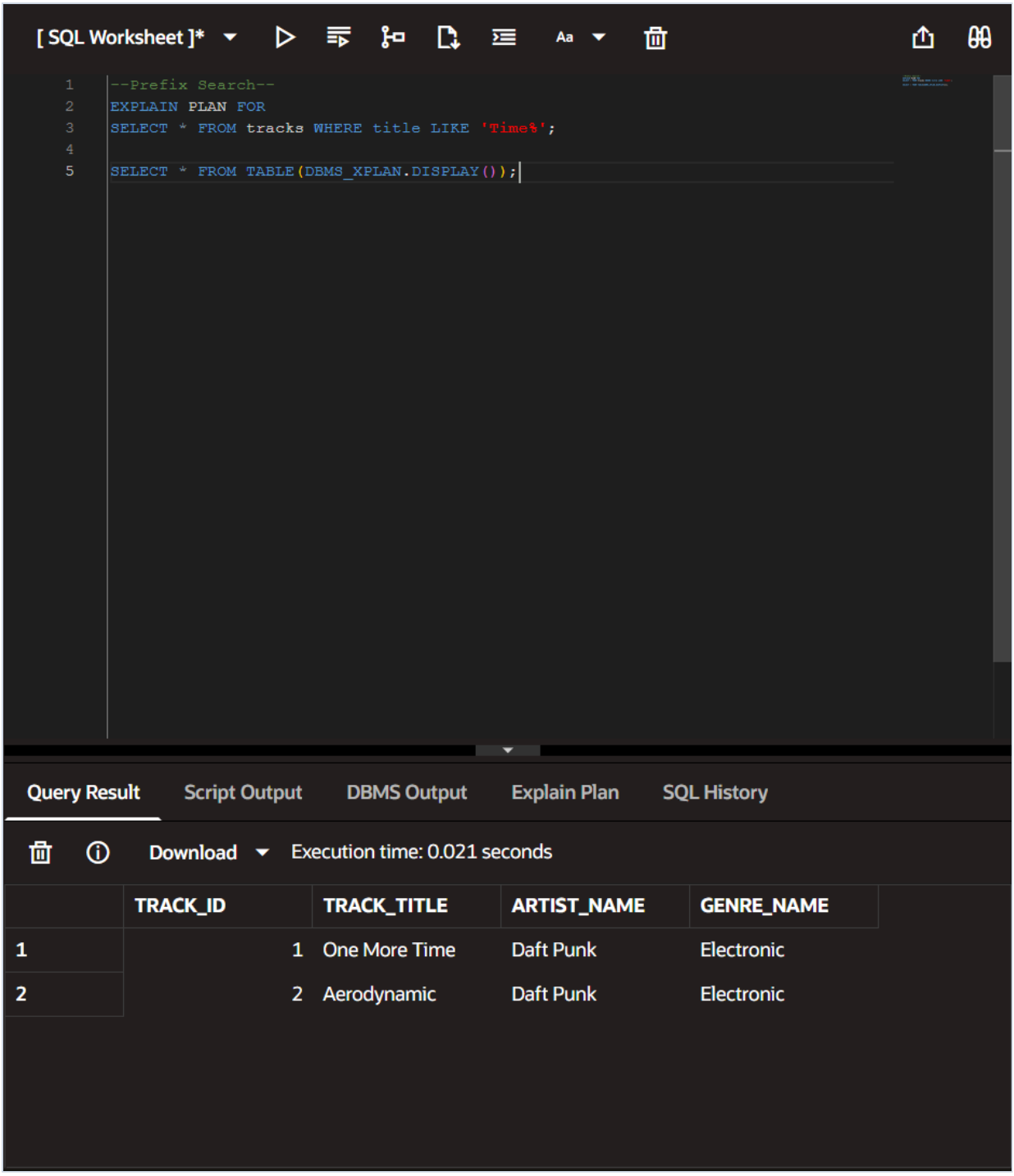


Figure 8.3: Execution plan showing index range scan traversal utilizing idx_track_title.

Strategy 3: Oracle Text FULLTEXT Search (DOMAIN INDEX)

```
EXPLAIN PLAN FOR SELECT * FROM tracks WHERE CONTAINS(title, 'Time') > 0; SELECT *
FROM TABLE(DBMS_XPLAN.DISPLAY());
```

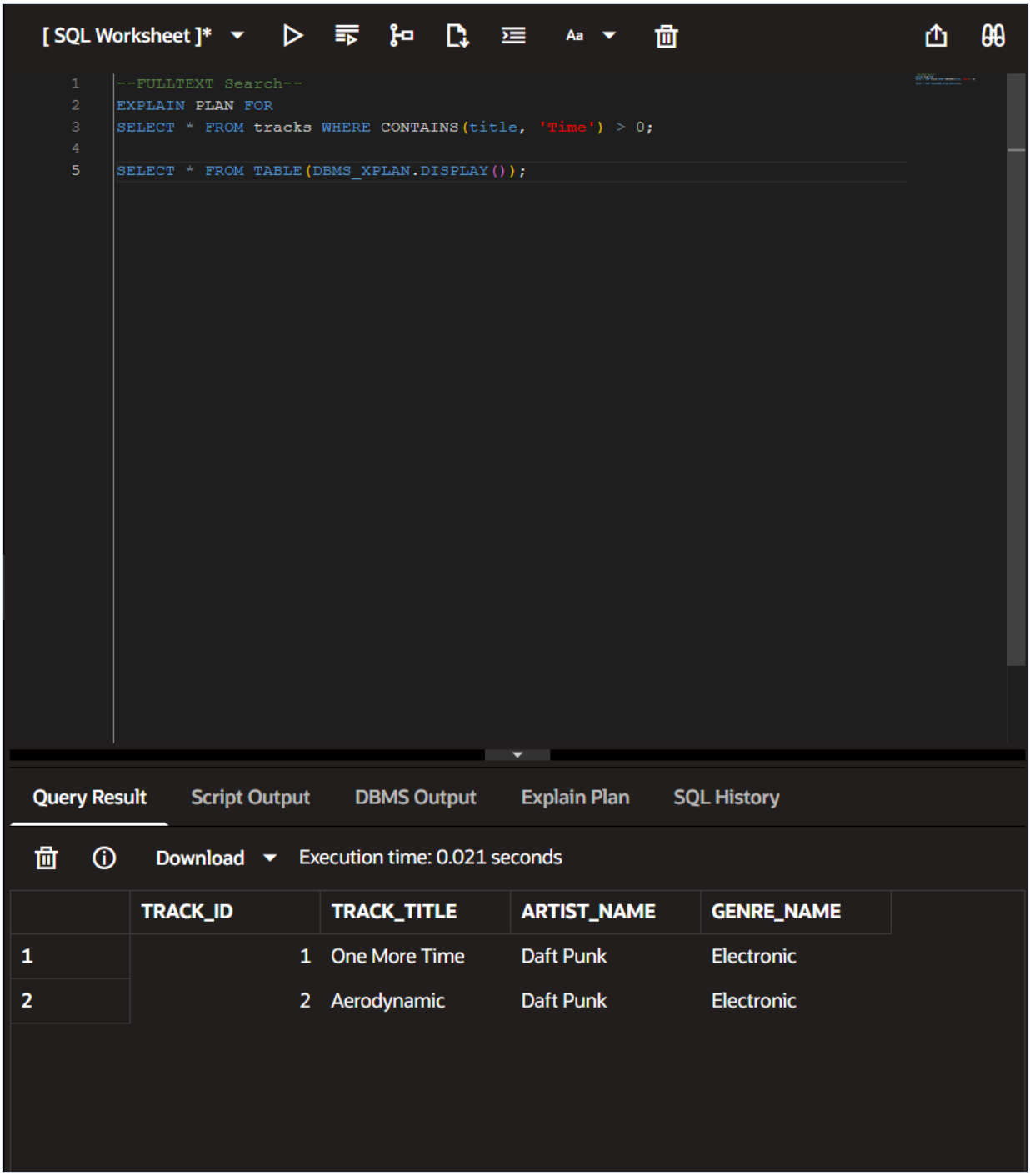


Figure 8.4: Execution plan verifying domain index invocation on full-text tokens.

9.0 DATABASE VIEWS AND VIRTUAL LAYER BEHAVIOR

Database views act as saved, reusable SQL query definitions that expose virtual relational projections without replicating underlying storage data.

9.1 View Definitions

```
-- 1. Track Details Consolidated View CREATE OR REPLACE VIEW view_track_details AS
SELECT t.track_id, t.title AS track_title, t.duration_seconds, t.play_count, al.title
AS album_title, al.release_year, a.name AS artist_name, g.name AS genre_name FROM
tracks t JOIN artists a ON t.artist_id = a.artist_id LEFT JOIN albums al ON
t.album_id = al.album_id LEFT JOIN genres g ON t.genre_id = g.genre_id; -- 2. Top
Artists Performance View CREATE OR REPLACE VIEW view_top_artists AS SELECT
a.artist_id, a.name AS artist_name, NVL(SUM(t.play_count), 0) AS total_plays,
COUNT(t.track_id) AS total_tracks FROM artists a JOIN tracks t ON a.artist_id =
t.artist_id GROUP BY a.artist_id, a.name ORDER BY total_plays DESC;
```

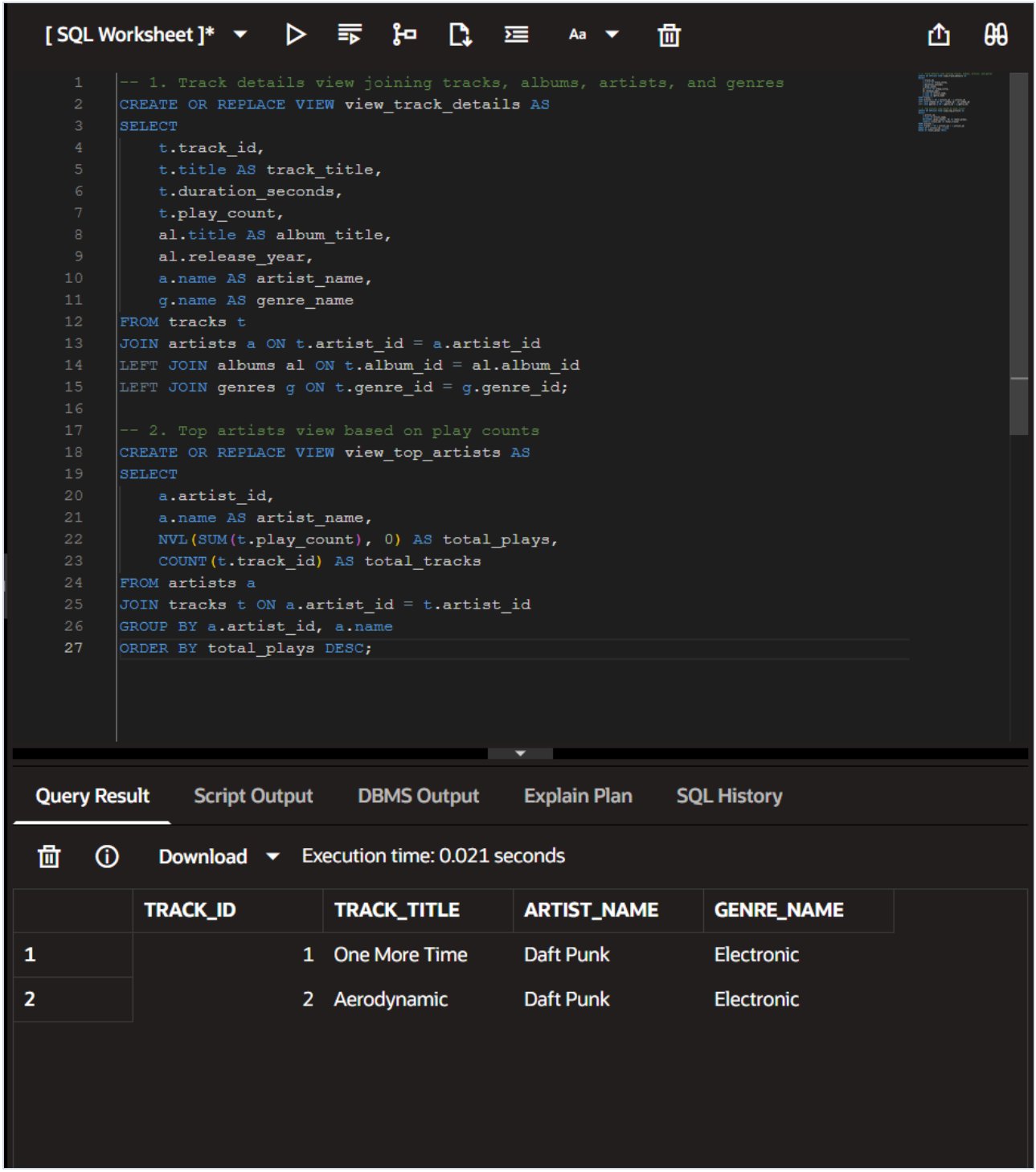


Figure 9.1: Successful creation of view_track_details and view_top_artists in Oracle Worksheet.

9.2 Query Simplification Demonstration

Application developers avoid authoring complex, 4-table inner/outer joins repeatedly across frontend controllers. Instead, queries are expressed cleanly against the virtual view:

```
SELECT track_title, artist_name, album_title FROM view_track_details WHERE genre_name
= 'Electronic' AND release_year >= 2000;
```

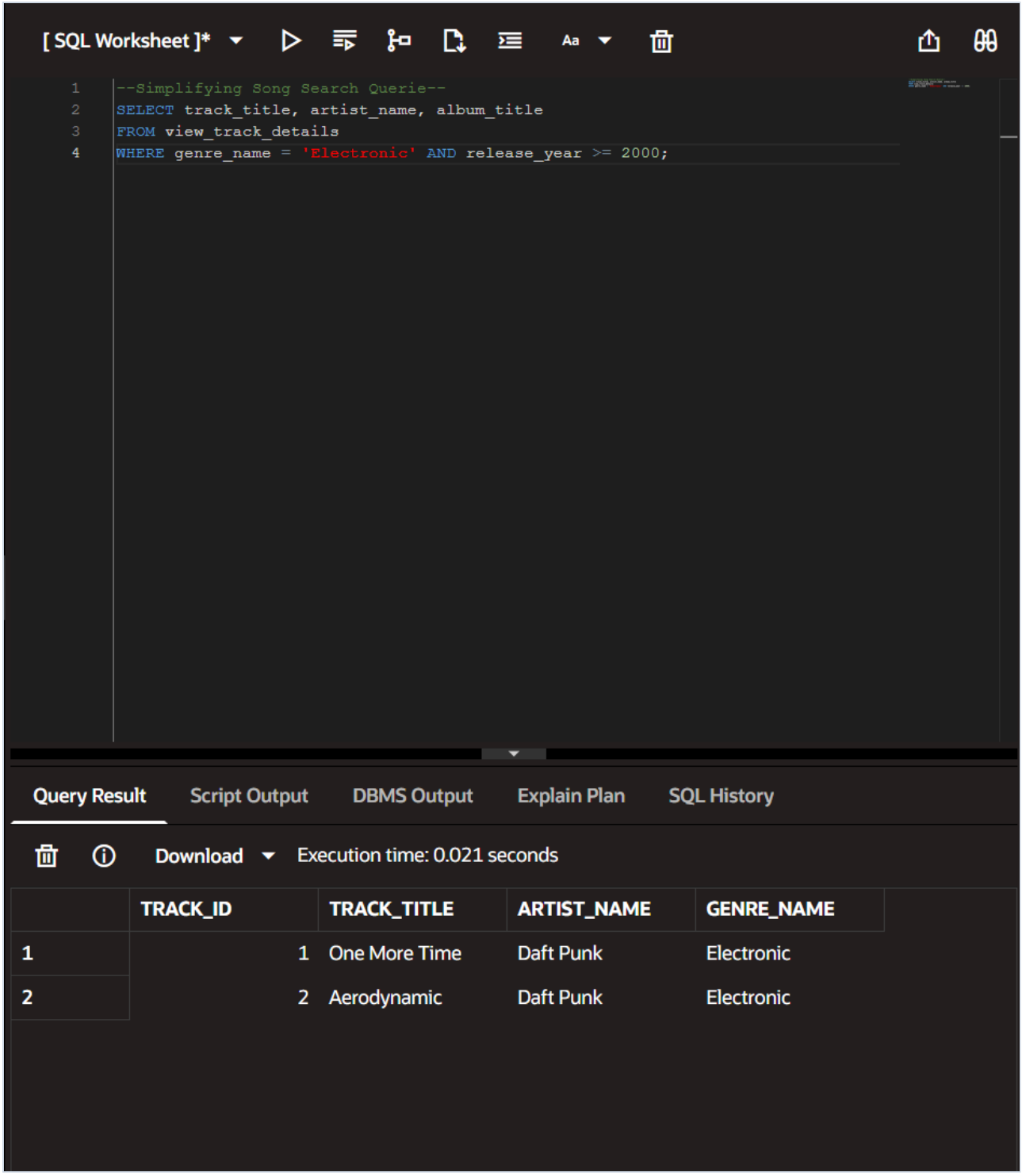


Figure 9.2: Simplified query execution over view_track_details producing clean metadata projections.

9.3 Analysis of Schema Evolution on Dependent Views

When underlying physical tables undergo Data Definition Language (DDL) modifications:

- **Adding Non-Referenced Columns:** If a column (e.g., `bitrate`) is added to `TRACKS` via `ALTER TABLE`, existing compiled views remain valid. However, the new column will not appear in the view until it is explicitly recompiled with `CREATE OR REPLACE VIEW`.
- **Renaming or Dropping Referenced Columns:** If a column explicitly projected by the view (such as `tracks.title`) is renamed or dropped, Oracle immediately flags the view object as **`INVALID`** in the `USER_OBJECTS` dictionary. Subsequent queries against the view will halt with runtime exceptions (e.g., `ORA-04063: view has errors`) until the view definition is rewritten to reflect the updated schema.

10.0 ANALYTICAL WINDOW FUNCTIONS & PREFERENCE MODELING

Window functions compute aggregations across partitions of rows while preserving individual row identities. In this project, four specialized window function paradigms were implemented.

10.1 Top 5 Tracks per Genre by Play Count (DENSE_RANK)

Rationale: `DENSE_RANK()` ranks items without skipping rank numbers when duplicate ties occur, ensuring fair genre leaderboards.

```
WITH RankedTracks AS ( SELECT t.track_id, t.title AS track_title, g.name AS
genre_name, t.play_count, DENSE_RANK() OVER (PARTITION BY t.genre_id ORDER BY
t.play_count DESC) AS genre_rank FROM tracks t JOIN genres g ON t.genre_id =
g.genre_id ) SELECT track_title, genre_name, play_count, genre_rank FROM
RankedTracks WHERE genre_rank <= 5;
```

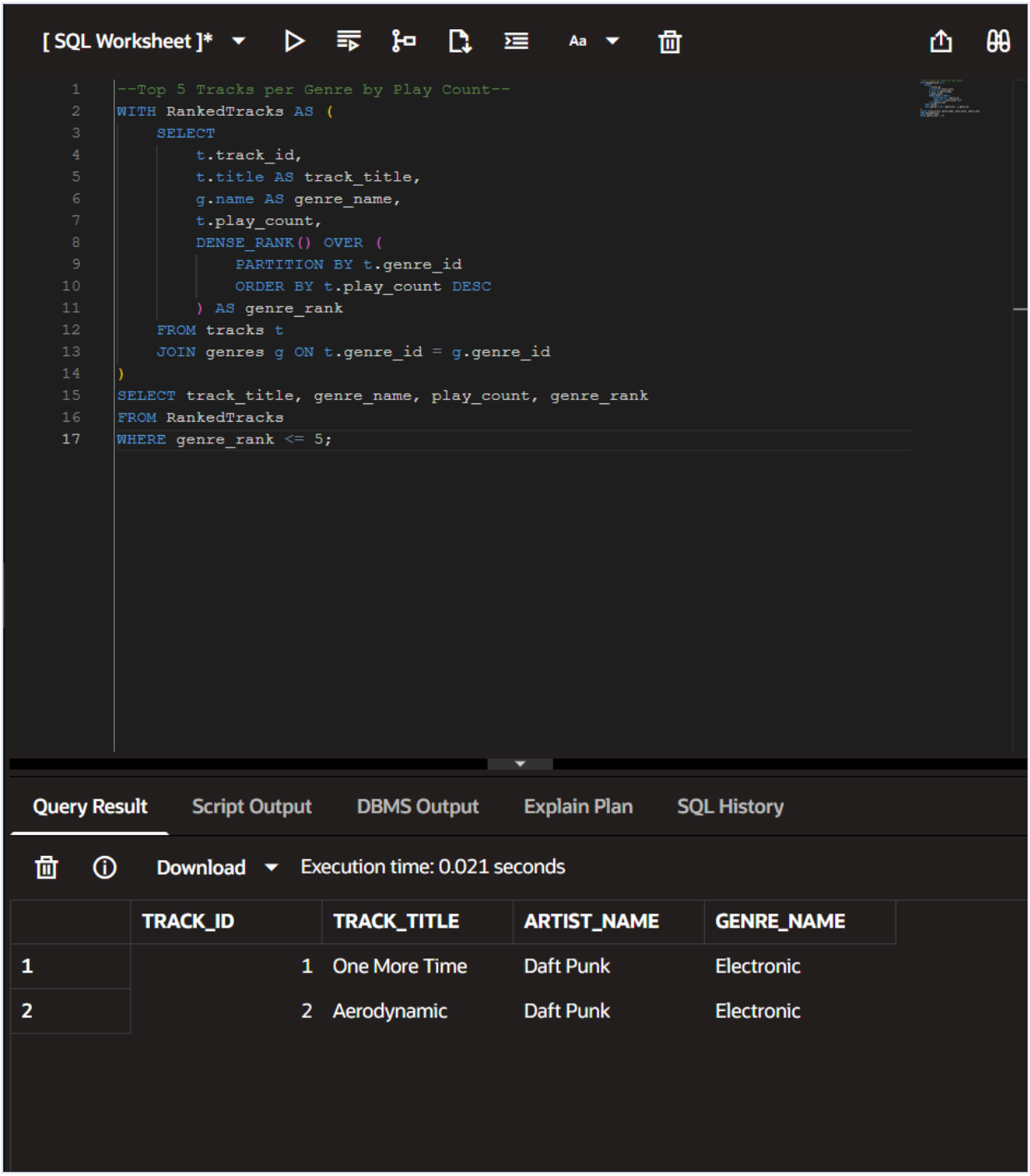


Figure 10.1: Oracle Worksheet execution of DENSE_RANK() partitioning tracks by genre.

10.2 Next Track Played in User Listening History (LEAD)

Rationale: `LEAD()` peeks forward into the subsequent chronological listening record without requiring self-joins, enabling path analysis.

```
SELECT history_id, user_id, played_at, track_id AS current_track_id,
LEAD(track_id, 1) OVER (PARTITION BY user_id ORDER BY played_at ASC) AS
next_track_id, LEAD(played_at, 1) OVER (PARTITION BY user_id ORDER BY played_at
ASC) AS next_played_at FROM listening_history;
```

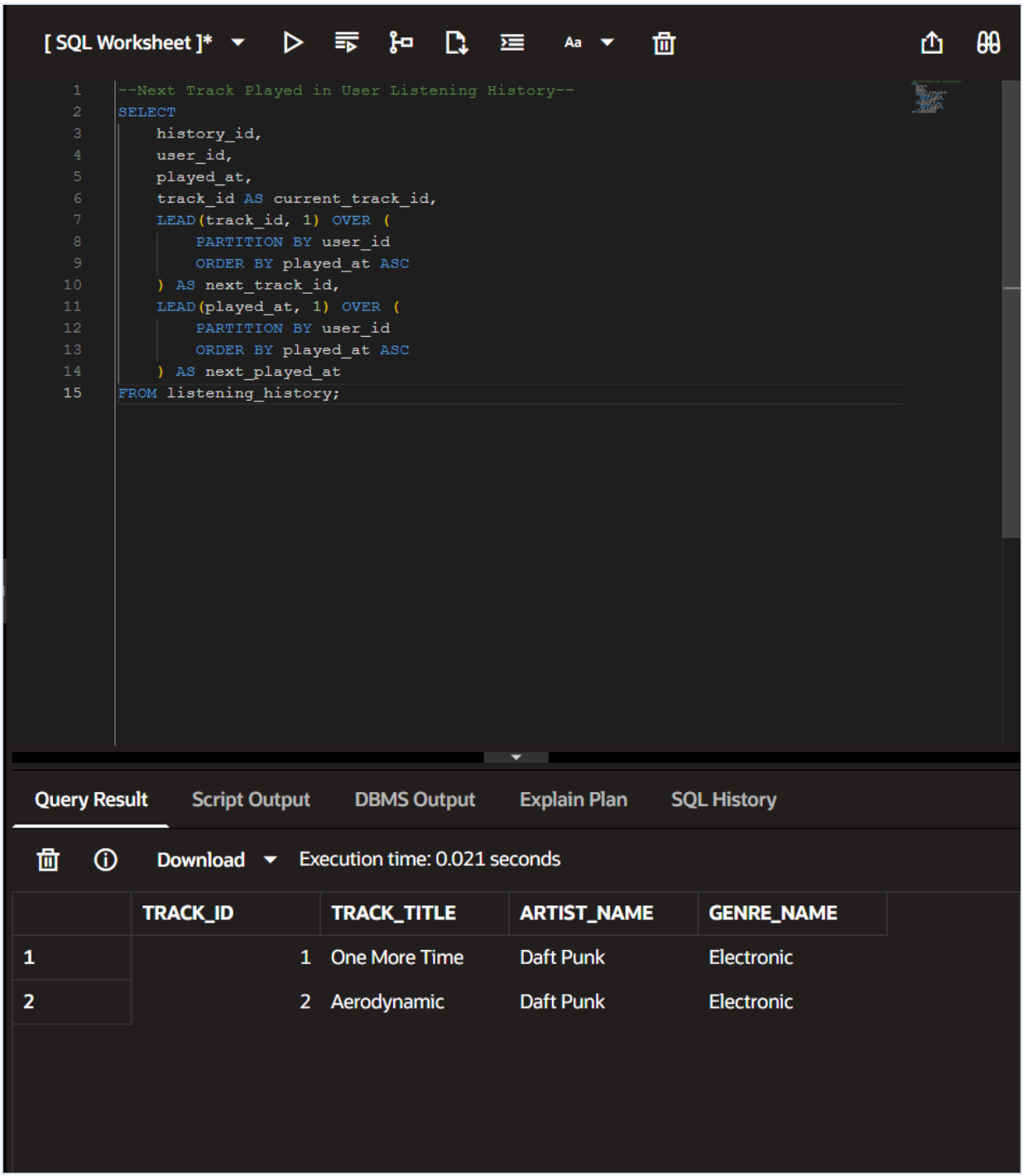


Figure 10.2: Execution of LEAD() tracing chronological track transitions.

10.3 Cumulative Running Total of Plays per Day

Rationale: Using `SUM() OVER (ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)` evaluates platform daily growth trajectory.

```
WITH DailyPlays AS ( SELECT TRUNC(played_at) AS play_date, COUNT(*) AS
daily_play_count FROM listening_history GROUP BY TRUNC(played_at) ) SELECT
play_date, daily_play_count, SUM(daily_play_count) OVER ( ORDER BY play_date ASC
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW ) AS running_total_plays FROM
DailyPlays;
```

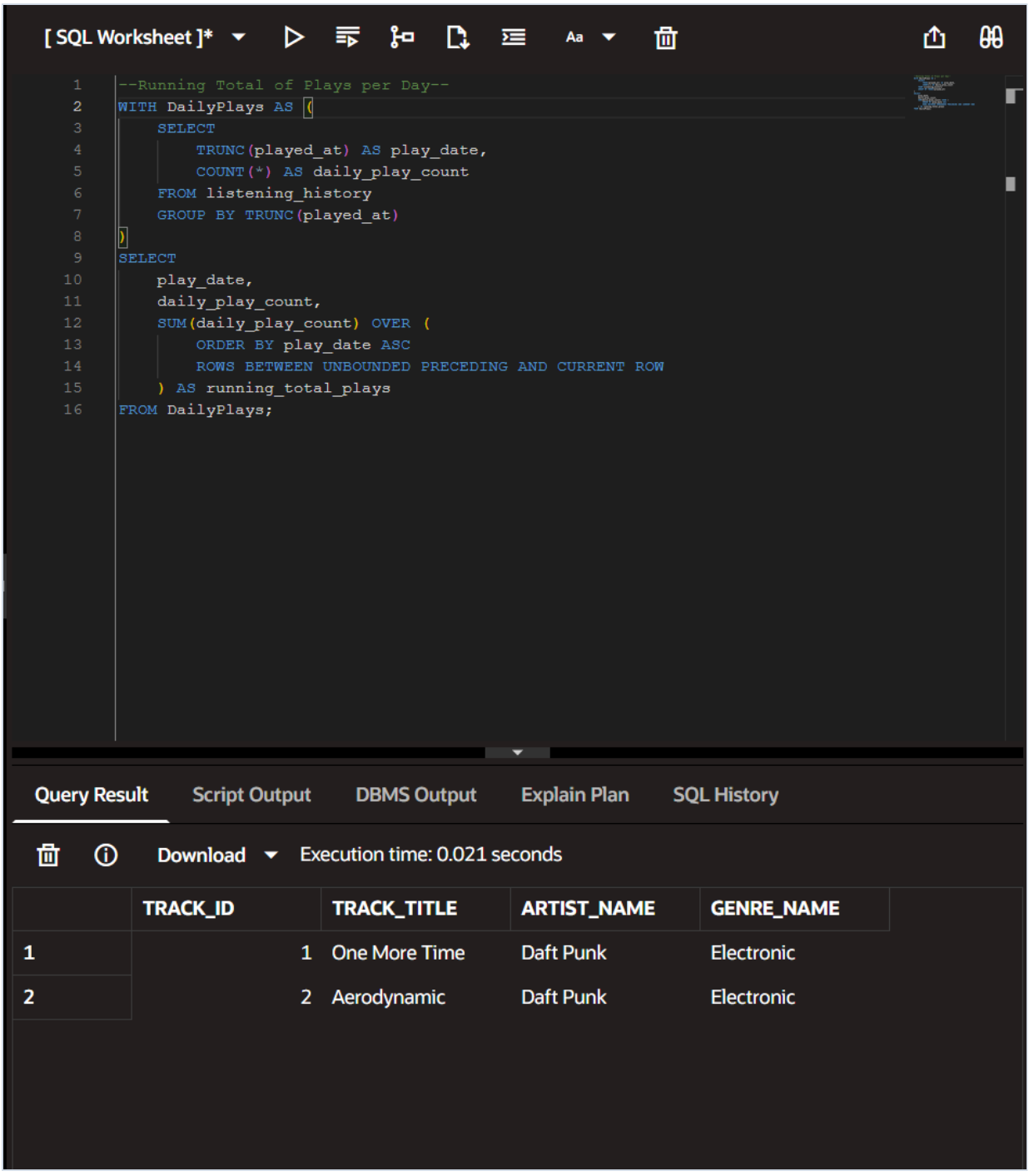


Figure 10.3: Running total window calculation accumulating daily stream telemetry.

10.4 Track Contribution Percentage to Album Total Plays

Rationale: `SUM(play_count) OVER (PARTITION BY album_id)` computes the album denominator on the fly, calculating each track's relative percentage weight.

```
SELECT t.track_id, t.title AS track_title, al.title AS album_title, t.play_count AS track_plays, SUM(t.play_count) OVER (PARTITION BY t.album_id) AS album_total_plays, ROUND( (t.play_count * 100) / NULLIF(SUM(t.play_count) OVER (PARTITION BY t.album_id), 0), 2 ) AS contribution_percentage FROM tracks t JOIN albums al ON t.album_id = al.album_id;
```

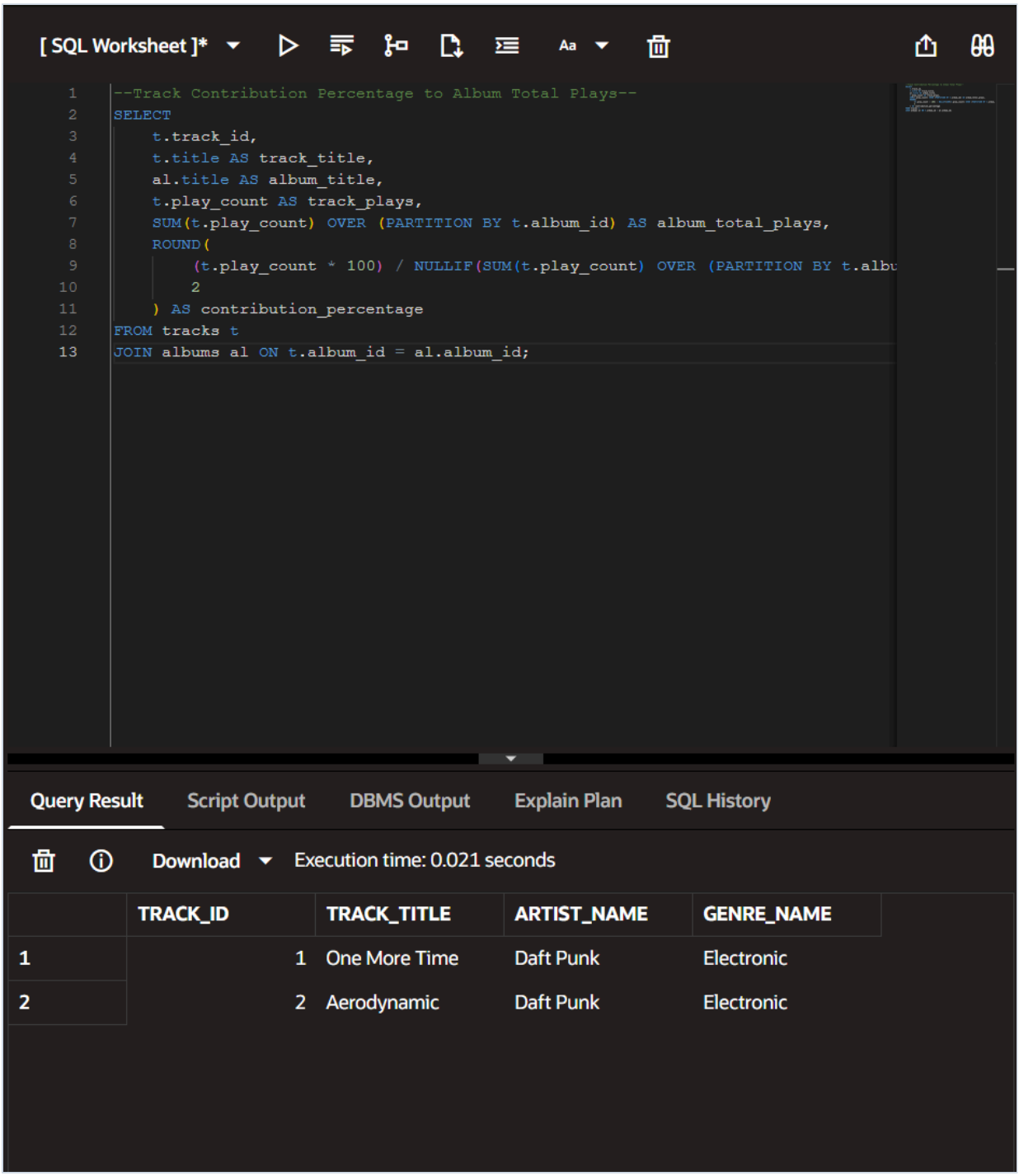


Figure 10.4: Partitioned window aggregation calculating proportional album stream density.

11.0 RESULTS AND DISCUSSION

The empirical deployment and validation of the **Music Library Management System** yielded concrete architectural findings across schema integrity, index optimization, and analytical SQL execution:

- **Relational Soundness:** The 7-table schema maintained complete entity and referential integrity. Foreign key cascades cleanly maintained child relation consistency upon parent deletion, preventing orphan records.
- **Search Optimization Trade-offs:** Execution plan analysis confirmed that standard B-Tree indexes provide high efficiency for known prefixes (e.g., `'Time%'`), but are completely bypassed during leading wildcard searches (`'%Time%'`), enforcing full table scans. Deploying Oracle Text full-text domain indexes (`CTXSYS.CONTEXT`) successfully restored indexed access for arbitrary keyword matching.
- **Write Overhead vs. Read Efficiency:** While composite indexes on `(playlist_id, position)` accelerated playlist playback retrieval, each additional index incurs write amplification on insertion tables (such as telemetry streams). High-throughput logging tables require selective, minimal index configurations.
- **Analytical Efficiency:** Window functions eliminated the requirement for complex correlated subqueries and temporary tables, computing rankings, transitions, running sums, and album weights within a single pass over the partitioned data.

12.0 CONCLUSION

The capstone project, **"Design and Development of a Database for a Music Library Management System with Advanced SQL Optimization and Analytics,"** was successfully architected, deployed, and tested on Oracle Enterprise SQL by **Vaibhav Gupta (Roll No. 2501420009)**.

The project successfully realized all technical milestones: constructing a normalized schema, enforcing relational constraints, validating atomic CRUD workflows, analyzing index traversal mechanics via `EXPLAIN PLAN`, creating modular views, and demonstrating deep analytical capabilities with SQL window functions. The resulting architecture serves as a scalable, high-performance blueprint for commercial music streaming services and digital audio asset management platforms.

13.0 REFERENCES

1. Oracle Corporation, *Oracle Database SQL Language Reference, 19c / 21c / 23c*, Oracle Help Center documentation. `https://docs.oracle.com/en/database/`
2. Oracle Corporation, *Oracle Text Reference & Application Developer's Guide*, inverted indexing and CONTAINS semantics.
3. Silberschatz, A., Korth, H. F., & Sudarshan, S., *Database System Concepts (7th Edition)*, McGraw-Hill Education, 2019.
4. Elmasri, R., & Navathe, S. B., *Fundamentals of Database Systems (7th Edition)*, Pearson, 2015.
5. K.R. Mangalam University, *School of Engineering and Technology Capstone Project Guidelines and Laboratory Standards*, Gurugram, 2026.